Neurocomputing 609 (2024) 128471

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

# Reward shaping via expectation maximization method

Zelin Deng [a], Xing Liu [b], Yunlong Dong [c,*]

[a] School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, China
[b] School of Mechanical Science and Engineering, Huazhong University of Science and Technology, China
[c] Department of Automation, Tsinghua University, China

ARTICLE INFO

Communicated by D. Macciò

*Keywords:*
Reinforcement learning
Reward shaping
Expectation maximization

ABSTRACT

Reinforcement Learning (RL) has found widespread applications in various decision making tasks. However, it still faces challenges such as the deadly triad, slow convergence, and rewards drop, which limit its practical scope. This paper primarily addresses the issues of slow convergence and rewards drop observed during RL training. Our proposed solution involves a reward shaping method composed of two distinct components, each serving a specific purpose: speeding up training and enhancing stability. These two components are interconnected through hyper-parameters, and it has been observed that the choice of hyper-parameters plays a critical role in determining the final performance of the RL algorithm. To optimize these hyper-parameters effectively, we employ a discrete sampling technique to cover the value ranges of these parameters. This discrete sampling creates a sparse set of data points within the reward matrix. Subsequently, we introduce a fitting approach based on the Expectation Maximization (EM) algorithm to estimate the global maximum of the reward matrix along with the corresponding hyper-parameters combination. This EM method significantly reduces computational complexity. Our extensive experimental results across various RL environments have demonstrated the effectiveness of our proposed method. It successfully mitigates the issue of rewards drop while simultaneously accelerating the convergence speed of the RL algorithm.

## 1. Introduction

Reinforcement learning (RL) operates through a trial-and-error mechanism, wherein it interacts with the environment, learns from these interactions, and strives to optimize its performance by maximizing the cumulative rewards received, ultimately aiming to acquire the optimal strategy. In recent years, with the development of RL, especially the methods combined with Deep Learning, have been successfully applied in various fields, such as playing games [1], autonomous driving [2], image processing [3] and quantitative analysis [4], etc. Deep RL combines the perception ability of deep learning and the decision making ability of RL to solve high-dimensional decision making problems [5]. Although RL has been successfully applied in many applications, there are still some prominent issues that limit its scope of application, such as: slow convergence speed, the curse of dimensionality [6], rewards drop [7], and deadly triad [8], etc. This paper focuses on slow convergence speed and rewards drop in the training process. Many researchers have proposed various improvement methods for the slow training speed issue in RL, among which reward shaping [9] is the most well-known. Reward shaping involves modifying the reward function by incorporating a potential function. The goal of reward shaping is to ensure training convergence without altering the optimality of the learned policy. Theoretical proofs exist to demonstrate that shaping the reward using a potential function does not change the optimal policy. Furthermore, extensive experiments have confirmed that appropriate initialization of the potential function can result in a significant acceleration of the training process in RL. To aid in the selection of an appropriate potential function, the work presented in [10] introduces a Lyapunov stability theory based approach known as LyapRL. This approach aims to determine the potential function and demonstrates its effectiveness in expediting the training process in RL. Notably, the successful application of LyapRL in the field of robotics, as discussed in [11], provides strong evidence of its effectiveness in practice.

The training process in RL can often exhibit instability, and in their work [7], the authors investigate a phenomenon known as rewards drop during training. To address this issue and enhance the overall stability of RL training, they propose a method called Variance Aware Reward Shaping (VAR). To further improve the training performance of RL, this paper combines two approaches, LyapRL and VAR, to achieve faster convergence while mitigating rewards drop during training. This combination is achieved by introducing two hyper-parameters that serve to blend the shaped terms from both LyapRL and VAR. The careful

* Corresponding author.
*E-mail address:* yunlongdong@mail.tsinghua.edu.cn (Y. Dong).

https://doi.org/10.1016/j.neucom.2024.128471
Received 22 September 2023; Received in revised form 13 July 2024; Accepted 25 August 2024
Available online 28 August 2024
0925-2312/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

selection of these hyper-parameters becomes crucial in achieving the desired performance.

In machine learning, hyper-parameters optimization is often carried out using a random search strategy [12,13], but this approach is inefficient and challenging to apply in computationally complex or data-intensive scenarios. There are also related works [14,15] that perform hyper-parameters optimization by empirically updating hyper-parameters within specific ranges, but these methods heavily rely on prior knowledge and may not be widely applicable. Works based on intelligent optimization algorithms [16], such as simulating particle swarm, genetic algorithms, and ant colonies, searching for hyper-parameters and can achieve good results. However, these methods introduce additional computational overhead and may not be suitable for RL algorithms that have significant computational resource requirements.

This paper introduces a method for representing the influence of hyper-parameters on RL training using a reward matrix. By discretely sampling the value ranges of hyper-parameters, a sparse set of points is constructed in the reward matrix. Subsequently, a fitting approach based on the Expectation Maximization (EM) algorithm is proposed to estimate the global maximum of the reward matrix and the corresponding hyper-parameters combination. This combination serves as the final choose. The proposed EM method significantly reduces the computational complexity by fitting the sparse point set in the reward matrix. Experimental results demonstrate that the reward matrix converges with only a small number of iterations, making it suitable for RL, which demands substantial computational resources.

In summary, this paper presents a strategy that leverages both LyapRL and VAR, utilizing specific hyper-parameters to balance their contributions and ultimately achieve improved RL training performance with faster convergence and reduced rewards drop. The main contributions of this paper are stated as follows:

1. A reward shaping method is proposed to organically combine LyapRL and VAR. This method has the dual advantage of accelerating the RL process while also improving stability to mitigate the problem of rewards drop.
2. An EM based method is proposed to effectively search the optimal hyper-parameters with only a small number of iterations and largely reduce the computational cost.
3. We applied the proposed algorithm to the task of dexterous hand turning valves, verifying that the introduced reward shaping method can significantly enhance training stability in complex control tasks.

The remainder of this paper is structured as follows: In Section 2, we present essential preliminaries and introduce relevant notations to establish a foundation for the subsequent discussions. Section 3 outlines the proposed reward shaping method and introduces the EM based hyper-parameters searching approach. Section 4 offers a demonstration of the effectiveness of the proposed method through a series of experiments. Finally, in Section 5, we provide a summary of the work presented in this paper and draw conclusions based on our findings. We strive to maintain a consistent structure throughout the paper for clarity and ease of navigation.

## 2. Preliminaries

According to [8], RL can be considered as a Markov decision process (MDP). A MDP is expressed as a tuple $(S, \mathcal{A}, T, R, \gamma)$, where $S, \mathcal{A}$ represent the sets of actions and states, respectively. $T(s'|s,a)$ represents the probability of transition from the state $s$ to the state $s'$. $R(s,a,s')$ is the reward obtained when the action $a$ is taken. Here, $\gamma$ is the discount factor for the rewards. $\pi(a|s)$ represents the policy, which is the mapping from the state to the action to be taken. The goal of the RL algorithm is to find an optimal policy $\pi^*(a|s)$ that can maximize the sum of rewards $\sum_{t=0}^{\infty}\gamma^t R_t(s_t,a_t,s_{t+1})$. The value function is defined as:

$$V_\pi(s) \triangleq \mathbb{E}_{a_t\sim\pi(a_t|s_t)}[\gamma^t R_t(s_t,a_t)|s_0=s]. \tag{1}$$

The action value function is:

$$Q_\pi(s,a) \triangleq \mathbb{E}_{a_t\sim\pi(a_t|s_t)}[\gamma^t R_t(s_t,a_t)|s_0=s,a_0=a]. \tag{2}$$

And the advantage function is defined as:

$$A_\pi(s,a) \triangleq Q_\pi(s,a) - V_\pi(s). \tag{3}$$

It is worth noting that the true values of $V_\pi(s)$ and $Q_\pi(s,a)$ are difficult to obtain, and neural networks are used to estimate them in Deep RL.

The Actor–Critic [17] is well-known and widely applied [18,19]. A "critic" uses a neural network with parameters $\theta$ to approximate the value function $V_\pi(s)$, and the target function of "critic" is defined as:

$$\mathcal{L}_\theta = \mathbb{E}_{t\sim D}\|G_t - V_\pi(s_t;\theta)\|_2^2, \tag{4}$$

where $D$ is the experience set, and $G_t = R_t + \gamma V_\pi(s_{t+1})$ is the TD-target [8]. Similarly, an "actor" applies a neural network with parameters $\phi$ to estimate the policy $\pi(a|s;\phi)$. $\phi$ is updated by minimizing the policy loss defined as:

$$\mathcal{L}_\phi = \mathbb{E}_{t\sim D}\left[\log\pi(a_t|s_t;\phi)\hat{A}_t\right], \tag{5}$$

where $\hat{A}$ is the estimated advantage function $A$. During the learning process, "critic" and "actor" alternately update their parameters $\theta$ and $\phi$ to obtain the optimal policy $\pi^*$.

In complex interactive environments where both the state space and action space have high dimensions, the convergence of RL tends to be slow. This slow convergence rate can be particularly pronounced in Deep RL, further limiting its practical applications. Addressing this challenge, extensive research efforts have been dedicated to finding a general and principled approach to expedite the convergence speed of RL training. One notable early contribution in this direction is presented in [9], which introduced a potential based reward shaping technique. This technique has the potential to significantly reduce the training time required for RL. It can be expressed as:

$$R'(s,a,s') \overset{\Delta}{=} R(s,a,s') + \gamma\Phi(s') - \Phi(s), \tag{6}$$

where $\Phi(s)$ is the potential function of state $s$. When the initial value of $\Phi(s)$ is the optimal value function $V^*(s)$, the training can be accelerated. However, in a complex environment, accurately estimating $V^*(s)$ can be a challenging task. The authors in [20] extend the action $a$ into the potential function, using the action value function to represent the potential function, which is summarized as:

$$R'(s,a,s',a') \overset{\Delta}{=} R(s,a,s') + \gamma\Psi(s',a') - \Psi(s,a) \tag{7}$$

Building upon these previous works, LyapRL [10] suggests utilizing $R(s,a)$ directly instead of $\Phi(s,a)$, since accurately estimating the value of states can be challenging. Through Lyapunov stability analysis, the authors have demonstrated that this method effectively accelerates the training speed of RL while preserving the optimal policy.

VAR [7] is aimed at mitigating the issue of rewards drop during the training process by utilizing a variance reduction approach. The authors introduce a time smoothing method given by $\beta\hat{R}_{t-1} + (1-\beta)R_t$, where the hyper-parameter $\beta$ is utilized to balance the weighting of rewards from two different time steps. Theoretical proofs provided by the authors establish that this method can effectively reduce variance in the later stages of training. Extensive experimental results align with these theoretical assumptions, demonstrating the effectiveness of this design in practice. An important feature of the VAR method is that it does not necessitate the modification of the value function, making it easily compatible with other RL algorithms. This property enhances its versatility and applicability in various RL settings.

## 3. Method

In this section, we will introduce our proposed reward shaping blending both LyapRL and VAR, and trade off the relationship between the hyper-parameters $\beta$ and $\lambda$.

### 3.1. Reward shaping

VAR focuses on solving the problem of rewards drop in the RL training process, and effectively improve the final performance. The reward trajectory $R_t$ in VAR is defined by:

$$\hat{R}_t = \beta \hat{R}_{t-1} + (1-\beta) R_t, \tag{8}$$

where $\hat{R}_t$ denotes the VAR trajectory, the hyper-parameter $\beta \in [0, 1)$ is the smoothing factor. $\hat{R}_t$ is usually initialized to 0 and is continuously updated in the subsequent process.

LyapRL speeds up the training process of RL by modifying the reward function, and the shaped reward function leads to a convergence guarantee and an invariant optimality policy. The reward shaping based on Lyapunov function is as follows:

$$R_t^{lyap} = R(s_t, a_t) + \lambda(\gamma R(s_{t+1}, a_{t+1}) - R(s_t, a_t)), \tag{9}$$

where $\lambda$ is a weighting factor of the shaped term $\gamma R(s_{t+1}, a_{t+1})$. It is worth noting that the modification of the reward function in Eqs. (8) and (9) are compatible, so the two equations can be combined to get our proposed reward shaping by:

$$\hat{R}_t = \beta \hat{R}_{t-1}^{lyap} + (1-\beta) R_t^{lyap} \tag{10}$$

The reward shaping proposed in this paper has $\beta$ and $\lambda$ two hyper-parameters. By adjusting the values of $\beta$ and $\lambda$, we can control the tradeoffs between accelerating training speed and improving stability. While utilizing VAR or LyapRL independently can enhance the training process, it is important to note that combining the two does not necessarily result in a simple additive effect. In other words, the VAR and LyapRL components in EMRL are not entirely independent of each other. To address this challenge and ensure that the reward shaping designed in this paper achieves its intended purpose, we will introduce a hyper-parameters estimation method based on EM in the next section. This method aims to optimize the combination of VAR and LyapRL within EMRL to maximize its effectiveness in accelerating training and suppressing rewards drop.

The details of the reward shaping function proposed in this paper are shown in Algorithm 1. And Section 4.1 will prove the effectiveness of the reward shaping proposed in this paper through experiments.

**Algorithm 1** $Q$-Learning with the proposed reward shaping

$\forall s \in \mathcal{S}, \forall a \in \mathcal{A}, \tilde{Q}^0(s,a) \leftarrow 0, \hat{R} \leftarrow 0, t \leftarrow 0$, $\beta$ and $\lambda$ are the hyper-parameters of VAR and LyapRL.
**while** not convergence **do**
  Sampling transition $(s_t, a_t, R_t, s_{t+1}, d_t)$ from the interacting with the environment, where $d_i$ indicates whether the training is finished or the trajectory is valid.
  $R_t^{lyap} \leftarrow R(s_t, a_t) + \lambda(\gamma R(s_{t-1}, a_{t-1}) - R(s_t, a_t))$
  $\tilde{Q}^{t+1}(s_t, a_t) \leftarrow (1-\alpha)\tilde{Q}^t(s_t, a_t) + \alpha\{R(s_t, a_t) + \lambda(\gamma R(s_{t-1}, a_{t-1}) - R(s_t, a_t)) + \gamma \max_{a \in \mathcal{A}} \tilde{Q}^t(s_{t+1}, a)\}$, where $\alpha$ is the learning rate.
  $t \leftarrow t+1$
  **if** $d_t = 1$ **then**
    $\hat{R} \leftarrow 0$
  **end if**
**end while**

### 3.2. EM based estimation method

Based on the experiments illustrated in Fig. 5, the results indicate the effectiveness of the reward shaping method proposed. However, it is worth noting that this method is notably sensitive to the choice of hyper-parameters. In experiments with appropriate hyper-parameters combinations, proposed reward shaping significantly enhances the training process. The hyper-parameters combinations identified through the search effectively suppress the phenomenon of rewards drop and exhibit faster learning efficiency. However, it is worth noting that certain combinations of hyper-parameters can have a detrimental effect on training performance, leading to some degree of side effects. Therefore, the careful selection of suitable hyper-parameters is of paramount importance for achieving the expected outcomes of proposed reward shaping method.

In order to be able to better describe the impact of parameters $\beta$ and $\lambda$ on the final reward, we construct a reward matrix $M \in \mathbb{R}^{W \times H}$. $W$ and $H$ denote several valid sets of values for $\beta$ and $\lambda$. The value $\hat{M}_{(i,j)}$ of matrix M on position $(i, j)$ represents the expected reward when $\beta = i, \lambda = j$. If we can know the value of each element in the reward matrix $\hat{M}$, we can get the most suitable hyper-parameters for the environment. However, the grid search of the matrix requires the time complexity of $O(W \times H)$, which is unacceptable. This paper proposes a hyper-parameters searching algorithm based on EM. Several hyper-parameters are selected in the environment and form the reward set $\{M_{(\beta_i,\lambda_i)} | i = 1, 2, \ldots, n\}$. We iteratively estimate the reward matrix based on the reward set, thereby reducing the space complexity.

EM is an iterative optimization algorithm used to find the maximum likelihood estimation for a given model. The iterative estimation process of EM can be divided into two primary steps: the Expectation (E) step and the Maximization (M) step. In the E-step, a function denoted as $h_\theta(\bullet)$ is created to compute the expected likelihood using the current parameters $\theta^t$. In the M-step, the algorithm computes updated parameters $\theta^{t+1}$ that maximize the expected likelihood obtained in the E-step. These E-step and M-step procedures are alternated iteratively until convergence is achieved. An advantageous feature of the EM algorithm lies in its relatively low computational complexity and minimal storage overhead. This implies that integrating the EM algorithm into the training of RL models should not substantially augment computational demands or storage requirements, rendering it a practical and feasible choice for enhancing RL training.

#### 3.2.1. E-step

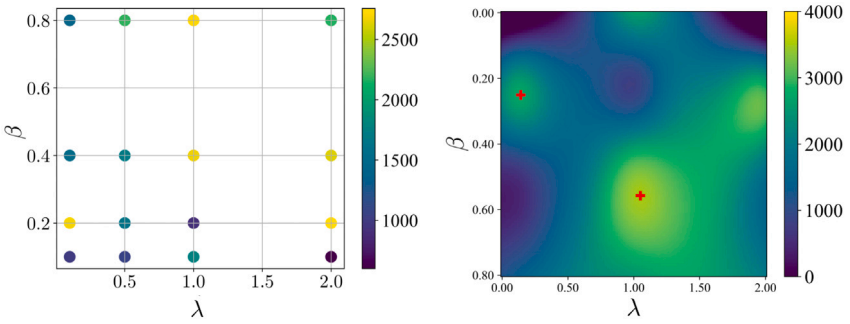
The goal of E-step is to construct a function $h_{\theta_t}(\cdot)$ with parameters $\theta_t$ to estimate the reward distribution. Firstly, we select $n$ sets of parameters through a sparse grid search, as shown in Fig. 1(a). And through experiments, we obtain the unbiased reward estimations for each hyper-parameters set:

$$\mathbb{S} = \{\frac{1}{m}\sum_{j=1}^{m} R^j_{(\beta_i,\lambda_i)} | i = 1, 2, \ldots, n\}, \tag{11}$$

where $R^j_{(\beta_i,\lambda_i)}$ represents the reward obtained in the $j$th experiment under hyper-parameters $(\beta_i, \lambda_i)$. To overcome the impact of experimental randomness, we use the average of rewards from $m$ experiments as the reward expectation for a given set of hyper-parameters. The Radial Basis Function (RBF) is selected as our estimation function, denoted as $h_{\theta_t}(\cdot)$, where $\theta_t$ represents the weight parameters of RBF at time $t$. If the set $\mathbb{S}$ contains $n$ elements, we can proceed to update the parameters in the RBF network using the experimental reward collection $\mathbb{S}$, which is summarized as:

$$\theta_{t+1} \leftarrow \arg\min_{\theta_t}(\frac{1}{n}\sum_{i=1}^{n}(h_{\theta_t}(\beta_i, \lambda_i) - \mathbb{S}_{(\beta_i,\lambda_i)})^2), \tag{12}$$

where $h(\theta_t, (\beta_i, \lambda_i))$ represents the RBF model's estimation of the rewards at time $t$ when the hyper-parameters are $(\beta_i, \lambda_i)$. $\theta_{t+1}$ represents the updated parameters after a single iteration, aiming to approximate




(a) Initialized sparse grid (b) Heatmap of the estimated reward matrix

**Fig. 1.** Initialization and estimation of the reward matrix. (a) 16 combinations of hyper-parameters are chosen to perform RL training to get the corresponding rewards and form the initial grid. (b) An estimated reward matrix is generated through E-step and the red cross arrows indicate the maximum points used in M-step.

the reward distribution $\mathbb{S}$. During the initial iteration for estimating the reward matrix, we perform discrete sampling for $W$ and $H$, each for $w$ and $h$ iterations, followed by subsequent fitting.

#### 3.2.2. M-step

Using the RBF model $h_{\theta_t}(\cdot)$, we traverse all possible parameter combinations to obtain an estimated reward matrix $\hat{M}^t \in \mathbb{R}^{W\times H}$. The estimated reward matrix $\hat{M}_t$ serves as an approximation by the estimation function $h_{\theta_t}(\cdot)$ to the ground-truth reward matrix $M$, as shown in Fig. 1(b). We identify the maximum value in the estimated reward matrix $\hat{M}^t$ and store both the maximum value and its corresponding hyper-parameters in the collection $\mathbb{P}$ to obtain:

$$\mathbb{P}_t = \{(\beta_i, \lambda_i, \hat{M}^t_{(\beta_i,\lambda_i)}) | i = 1, 2, \dots, n_t\}, \tag{13}$$

where $n_t$ represents the number of maximum values in the estimated reward matrix $\hat{M}_t$. Utilizing the hyper-parameters combinations from the set $\mathbb{P}$, conducting $m$ repeated experiments, and storing the obtained average rewards by:

$$\mathbb{S} \leftarrow \frac{1}{m}\sum_{j=1}^{m} R_{(\beta_i,\lambda_i)},\ (\beta_i, \lambda_i) \in \mathbb{P}_t \tag{14}$$

Next, we once again assess the accuracy of the estimated reward matrix, and the criteria are as follows:

$$(\frac{1}{n}\sum_{i=1}^{n}(h_\theta(\beta_i, \lambda_i) - \mathbb{S}_{(\beta_i,\lambda_i)})^2)^{\frac{1}{2}} \le \delta, \tag{15}$$

where $\delta$ represents the termination condition. If the condition in Eq. (15) is satisfied, we can obtain the final estimated reward matrix. Otherwise, we return to Eq. (12) to recalculate the model's weight and continue the iteration process. The global maximum value in the final estimated reward matrix and its corresponding hyper-parameters constitute the final optimal combination:

$$\beta^*, \lambda^* = \arg\max_{\beta,\lambda} \hat{M}^t_{(\beta,\lambda)} \tag{16}$$

Therefore, the computational complexity is reduced to $O(w \times h + n_{itr})$ through the proposed method, where $w \times h \ll W \times H$, where $n_{itr}$ is the number of iterations. Consequently, the proposed method can significantly reduce the computational complexity. Furthermore, subsequent experiments have demonstrated that the reward matrix typically converges in only a few iterations.

### 3.3. Iterative process

The proposed reward shaping approach, which combines the EM hyper-parameters estimation method, is referred to as EMRL. The overall iterative process of EMRL is illustrated in Fig. 2. EMRL can be divided into two distinct phases: the experimental phase and the estimation phase. In the experimental phase, we collect actual reward values obtained under different hyper-parameters settings, denoted as $\beta$ and $\lambda$. The estimation phase utilizes the actual rewards acquired during the experimental phase to estimate the reward matrix. Within the estimation phase, the candidate optimal hyper-parameters are determined by selecting the maximum value in the reward matrix, and this combination is then returned to the experimental phase. This iterative alternation between the two phases continues until an accurate estimation of the reward matrix is achieved.

The detailed iterative process is presented in Algorithm 2. We initiate this process by initializing the estimation matrix using a simple grid search. In this initial grid search, the step size is typically set to a large value to avoid excessive time consumption. Next, we select the maximum value in the estimation matrix as a candidate point and employ Algorithm 2 to perform experiments. The purpose of these experiments is to verify whether the estimated maximum value aligns with the experimental result. If the difference between the estimated maximum value and the experimental result exceeds a predefined termination condition, the iterative process continues. In Section 4, we will substantiate the effectiveness of the estimation method proposed in this paper through a series of experiments.

## 4. Experiments

In this section, we prove the effectiveness of the reward shaping method presented in this paper through a series of experiments. The experiments demonstrate that the proposed reward shaping method is notably sensitive to hyper-parameters. We further validate that the hyper-parameter selection method based on EM can efficiently select appropriate hyper-parameters.

### 4.1. The combination of LyapRL and VAR

In this section, our primary goal is to illustrate the effectiveness of combining LyapRL and VAR through a series of comprehensive experiments. As shown in Fig. 4, we choose the SD3 [21] as the baseline and subsequently integrate either LyapRL or VAR reward shaping methods into it. Observations from the experiments reveal several key insights. When applying LyapRL, we observe a significant acceleration in the training speed, but a noticeable rewards drop phenomenon still persists. On the contrary, the introduction of VAR mitigates the rewards drop issue but at the expense of a slower training process. Finally, we introduce our proposed EMRL method, which combines both LyapRL and VAR. Notably, this method not only expedites the training process but also effectively suppresses the rewards drop. To further investigate the impact of each component, we conduct ablation experiments, the results of which are presented in Table 1. We initiate experiments separately with LyapRL and VAR methods and then proceed with the EMRL method to assess their effects. The quantitative metrics include the maximum, minimum, and standard deviation observed in the final 20% of training steps. The quantitative results validate that the method

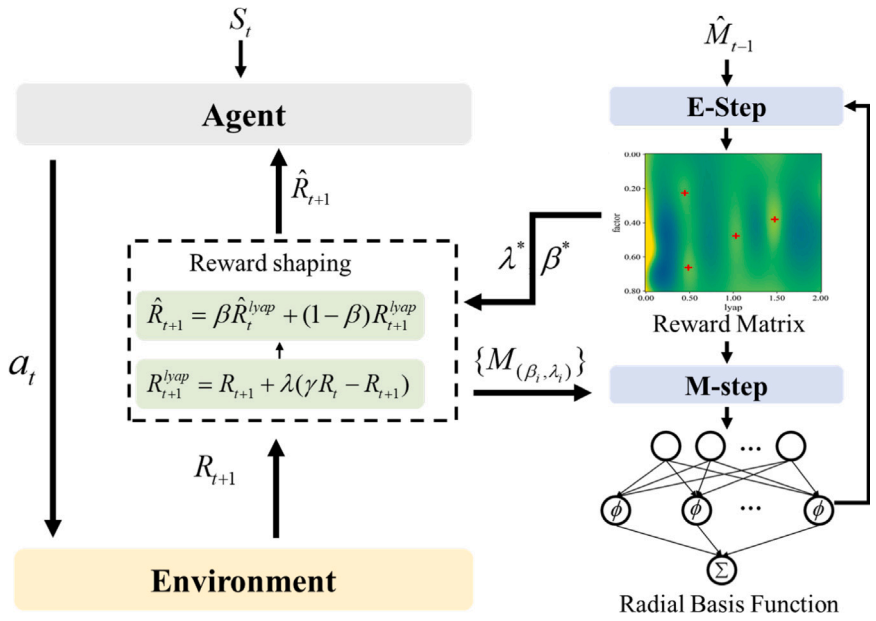



**Fig. 2.** Illustration of the proposed EMRL. The proposed reward shaping (left) can accelerate the model convergence speed and suppress the rewards drop in the training stage without changing the optimal policy. In order to overcome the problem that reward shaping is sensitive to hyper-parameters $\beta$ and $\lambda$, we propose a hyper-parameters searching algorithm based on EM (right). We estimate the reward Matrix and find the local maximum hyper-parameters in E-step, and update the parameters of the RBF model in M-step.

**Algorithm 2** Iterative estimation process

$\mathbb{B}^0$, $\mathbb{Y}^0$ are the initialization sets of $\beta$ and $\lambda$, respectively.
$\alpha_{(\beta,\lambda)}$ is the reward obtained when the hyper-parameters $\beta$ and $\lambda$ are used, and $\mathbb{A}$ is the sets of $\alpha_{(\beta,\lambda)}$.
$h(\mathbb{B}, \mathbb{Y}, \mathbb{A})$ is the estimation function.
$\mathbb{B}' \leftarrow \mathbb{B}^0$, $\mathbb{Y}' \leftarrow \mathbb{Y}^0, \mathbb{B} \leftarrow \emptyset, \mathbb{Y} \leftarrow \emptyset, \mathbb{A} \leftarrow \emptyset$
**while** not convergence **do**
  **for** $\beta, \lambda$ in $\mathbb{B}'$, $\mathbb{Y}'$ **do**
    $\bar{\alpha}_{(\beta,\lambda)} \leftarrow 0$, $n$ is the number of repeated experiments.
    **for** $i$ in $n$ **do**
      $\bar{\alpha}_{(\beta,\lambda)} \leftarrow \bar{\alpha}_{(\beta,\lambda)} + \frac{1}{n} \times \alpha^i_{(\beta,\lambda)}$, $\alpha^i_{(\beta,\lambda)}$ is the accumulated rewards in the $i$-th experiment.
    **end for**
    add $\bar{\alpha}_{(\beta,\lambda)}$ to the set $\mathbb{A}$.
    $\hat{M} \leftarrow h(\mathbb{B}, \mathbb{Y}, \mathbb{A})$, $\hat{M}_{(\beta,\lambda)}$ is estimated reward in position $(\beta, \lambda)$.
    find the global maximum and local maximum value set $\mathbb{A}'$ in matrix $\hat{M}$ and its corresponding coordinate position set
    add elements from set $\{\mathbb{B}', \mathbb{Y}', \mathbb{A}'\}$ to $\{\mathbb{B}, \mathbb{Y}, \mathbb{A}\}$.
  **end for**
**end while**
the estimated global optimal $\hat{\alpha}_{(\beta,\lambda)}$ is the maximum value in $\mathbb{A}$.

presented in this paper outperforms LyapRL in suppressing rewards drop during training and achieves faster convergence compared to VAR, aligning with our intended design objectives. The reward shaping and EM algorithm proposed by us can be applied not only in algorithms based on Q-learning but also in policy-based algorithms such as DDPG, with the results illustrated in Table 2.

It is worth noting that in the Hopper environment, the training results obtained using reward shaping in conjunction with the EM algorithm are slightly lower in terms of maximum reward compared to the method using only reward shaping. This is due to the strong randomness associated with the maximum value of rewards, while the average value of rewards is more indicative of the policy's ability in most situations. Consequently, our focus during parameter search is more on the average value of rewards. When not using the EM algorithm, the learned policy might achieve high rewards in a particular episode, but the average value and variance of rewards across all episodes are relatively poor. Therefore, we still consider the results obtained using the EM algorithm in the Hopper environment to be more superior.

### 4.2. Sensitivity experiment

We initially conduct experiments to validate the efficacy of our proposed reward shaping method. As shown in Fig. 5, we observe that the reward trajectory represented by the red curve exhibited higher rewards in the early stage (initial 20% of steps) compared to the green curve. Additionally, the red area in the later stages of training (final 20% of steps) is smaller than the green area. These results affirm that the EMRL method presented in this paper has the capability to expedite the convergence of the training process and alleviate the rewards drop. However, it is essential to acknowledge that not all combinations of hyper-parameters lead to improved training performance. This is evident in the case of the blue curve depicted in Fig. 5, where the final reward after applying the EMRL method is reduced. This underscores the critical importance of selecting appropriate hyper-parameters for the effectiveness of EMRL. Consequently, our next step involves the use of an EM based algorithm to systematically identify suitable hyper-parameters.

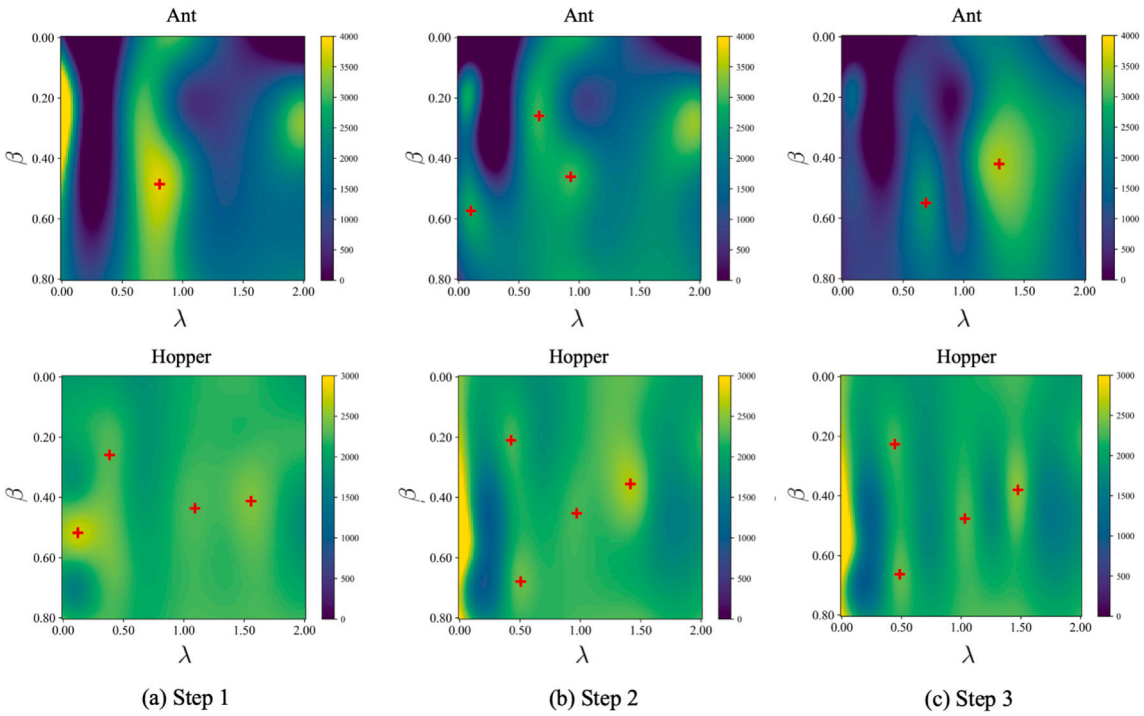



**Fig. 3.** Visualization of the iterative process of the estimated reward matrix in the Ant and Hopper environments. The red cross arrows represent the maximum points in the reward matrix. We will use the maximum point for the next round of the experiment to make the estimation process gradually converge. The proposed hyperparameter selection method was compared with randomly selected parameters and sparse grid search.

**Table 1**
The quantitative results of the experiments on five different random seeds. All the indicators are based on the last 20% steps during training. And std represents the standard deviation. The best results on the indicators are marked bolded. The results show that the hyper-parameters estimation method based on EM proposed in this paper can accurately select appropriate hyper-parameters.

| Reward shaping | EM | Ant | | | Hopper | | | Walker2D | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | mean | max | std | mean | max | std | mean | max | std |
| ✗ | ✗ | 1837 | 3000 | 937 | 2112 | 2564 | 558 | 1441 | 1958 | 551 |
| ✓ | ✗ | 2282 | 3144 | 897 | 2304 | **2831** | 522 | 1591 | 2111 | 492 |
| ✓ | ✓ | **2718** | **3307** | **597** | **2384** | 2702 | **339** | **1650** | **2154** | **487** |

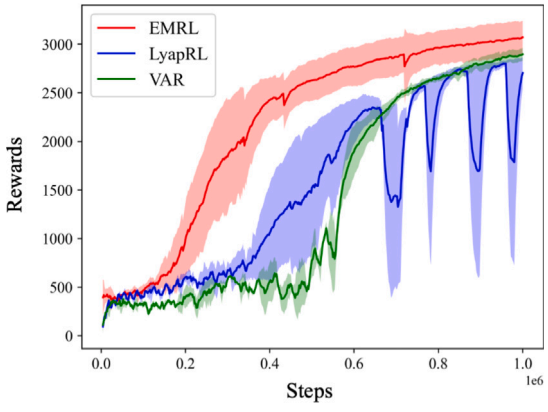



**Fig. 4.** The combination of LyapRL and VAR. We can see that the LyapRL method has a rewards drop in the final stage, while the training convergence speed of VAR is relatively slow. The method in this paper has the advantages of both, which proves the effectiveness of the combined method. The experiments are conducted with five different random seeds.

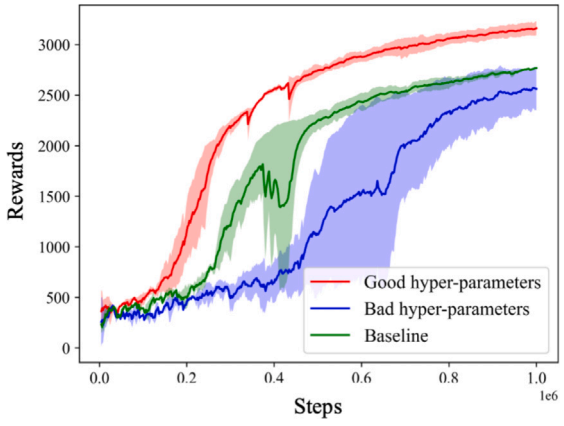



**Fig. 5.** The value of hyper-parameters $\lambda$ and $\beta$ have an important influence on RL training. Inappropriate hyper-parameters will lead to a serious decrease in the training performance. This fig shows the experimental results in the Ant environment. We have also observed the same phenomenon in the Hopper and Walker environments. The experiments are conducted with five different random seeds.

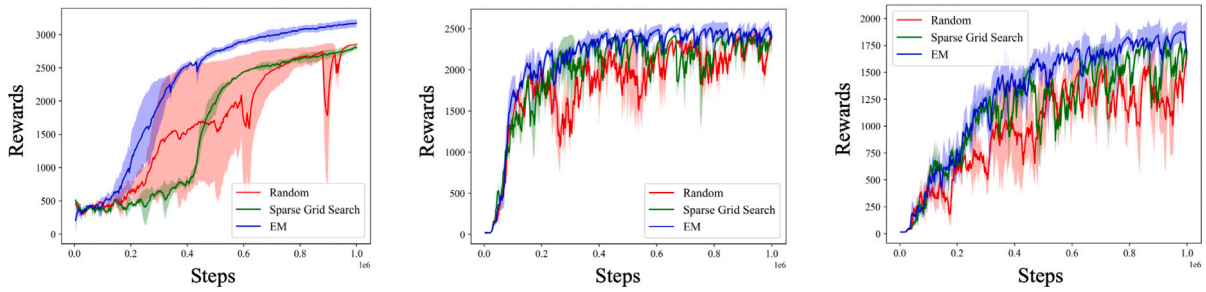



**Fig. 6.** The experiments on various environments (Ant, Hopper and Walker2D). We use the SD3 algorithm with LyapRL and VAR for training. In order to overcome the interference of random factors, all the experiments are conducted with five different random seeds. The red lines indicates using selected hyper-parameters. The solid lines represent the mean value and shaded area indicates the standard variance. The proposed hyperparameter selection method was compared with randomly selected parameters and sparse grid search.

**Table 2**
The quantitative results of the experiments on policy based method.

| Reward shaping | EM | mean | max | std |
|---|---|---|---|---|
| ✗ | ✗ | 1066 | 1786 | 392 |
| ✓ | ✗ | 1105 | 1950 | 549 |
| ✓ | ✓ | 1431 | 2054 | 342 |

### 4.3. Effectiveness analysis

To validate the effectiveness of the hyper-parameter selection method based on EM, we conduct experiments in the Ant and Hopper environments.[1] In these experiments, we utilize a large step interval (0.25 for $\beta$ and 0.5 for $\lambda$) to perform a two-dimensional grid search as the initialization of the reward matrix. The iteration process and the final reward matrix are visualized in Fig. 3. It is clearly evident that the reward matrix tends to converge after a few iteration steps. Subsequently, we conduct experimental tests using the largest maximum value in the reward matrix. Remarkably, we observe that the hyper-parameters obtained through the iterative method yield higher rewards, and the rewards drop in the final stage is alleviated. To further substantiate the effectiveness of the proposed EM based hyper-parameters searching, we compare the results with randomly selected hyper-parameters, as shown in Fig. 6. This comparison clearly demonstrates a significant improvement in training effectiveness achieved by the searched hyper-parameters, providing empirical evidence of the efficacy of our proposed method.

### 4.4. Application experiment

To further demonstrate the practical value of the reward shaping method proposed in this paper, we applied it to the task of a dexterous hand turning a valve. The experimental environment for the dexterous hand valve turning task is shown in Fig. 7. The objective of the task is to control the joint positions of a three-fingered dexterous hand to rotate the red valve as quickly as possible. The dexterous hand possesses eight degrees of freedom, and current traditional control methods are unable to utilize multi-degree-of-freedom dexterous hands for complex operational tasks. Therefore, learning the valve rotating strategy through a dexterous hand is of high practical value.

The training experimental results are shown in Fig. 7. Initially, RL without reward shaping also managed to learn the valve-turning strategy, indicating the rationality of the system's reward function and model structure design. However, the angle of valve rotation achieved by the strategy learned without reward shaping was relatively small. After incorporating the proposed reward shaping method, the learned strategy was able to achieve a faster valve rotation. The comparison of the reward functions with and without reward shaping is illustrated in Fig. 8. The reward trajectory with reward shaping is superior to that without it, indicating that the reward shaping method proposed in this paper can also enhance the stability of training in actual control tasks while suppressing the rewards drop.

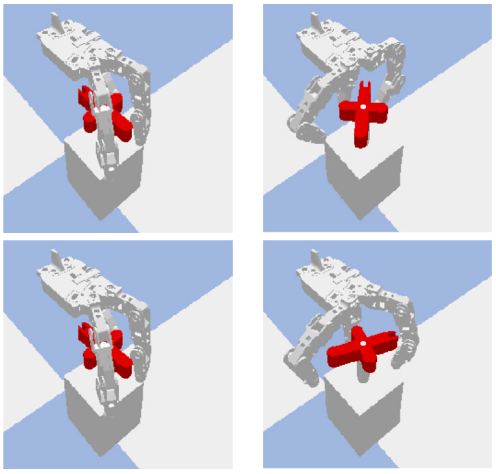



**Fig. 7.** The comparison results for the valve rotation task regarding the use of reward shaping. The first row and the second row, respectively, represent the policy trained without reward shaping and the policy trained with reward shaping.

### 4.5. Results discussion

Improving the training efficiency and reward values of reinforcement learning has long been a core direction of its improvement. We propose combining multiple improved reward shaping functions to enhance training stability while suppressing reward diminishment, and we have demonstrated the effectiveness of this method in experiments (as shown in Fig. 4). Moreover, the method is highly compatible and can be applied to two common types of reinforcement learning algorithms: policy-based and value-based (as shown in Table 2).

However, inappropriate combination parameters can not only fail to improve training stability but may also harm training accuracy (Fig. 5), making it necessary to design specialized algorithms to select the appropriate combination parameters. However, hyperparameter optimization for reinforcement learning is a highly challenging issue, as each reinforcement learning training session can last for several hours, and the time complexity introduced by classic parameter optimization algorithms, such as particle swarm optimization and genetic algorithms

[1] https://github.com/bulletphysics/bullet3

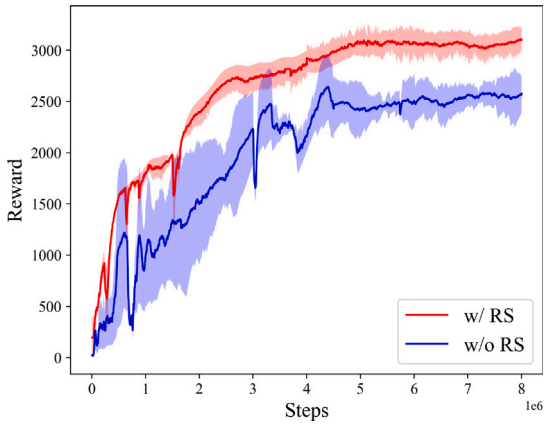



**Fig. 8.** The comparison of training results with (w/) and without (w/o) reward shaping (RS). A solid line indicates the mean, and a shaded area shows the variance from 10 repeated experiments using different random seeds.

is unacceptable. How to select hyperparameters for reward shaping within a reasonable time complexity is an intriguing issue. In this paper, we innovatively apply the Expectation Maximization algorithm to the hyperparameter optimization process, achieving more stable training outcomes and higher rewards through appropriate hyperparameter selection (as shown in Table 1).

## 5. Conclusions

In this paper, we introduce a novel reward shaping method designed to simultaneously expedite the training process in RL and mitigate the issue of rewards drop during training. In our experimental section, we substantiate the effectiveness of our method through both visualizing the trajectory of rewards and quantitatively comparing the final achieved performance. However, our experiments also reveal that the values of hyper-parameters $\lambda$ and $\beta$ exert a substantial influence on the training performance. To effectively find suitable hyper-parameters, we employ an efficient EM based searching algorithm. By discretely sampling the value ranges of hyper-parameters, a sparse set of points is constructed in the reward matrix. Subsequently, a fitting approach based on the EM algorithm is proposed to estimate the global maximum of the reward matrix and the corresponding hyper-parameter combination. Through an extensive series of experiments, we provide empirical evidence showcasing the effectiveness of our proposed EMRL method. Our results highlight its capacity to enhance RL training by achieving faster convergence and greater stability. However, the hyperparameter selection method proposed in this paper introduces additional time complexity. In subsequent work, we will propose more efficient parameter search algorithms to further improve the efficiency of the algorithm.

## CRediT authorship contribution statement

**Zelin Deng:** Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Xing Liu:** Visualization, Validation, Data curation. **Yunlong Dong:** Writing – review & editing, Supervision.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

No data was used for the research described in the article.

## References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.
[2] B.R. Kiran, I. Sobh, V. Talpaert, P. Mannion, A.A. Al Sallab, S. Yogamani, P. Pérez, Deep reinforcement learning for autonomous driving: A survey, IEEE Trans. Intell. Transp. Syst. 23 (6) (2021) 4909–4926.
[3] S.K. Zhou, H.N. Le, K. Luu, H.V. Nguyen, N. Ayache, Deep reinforcement learning in medical imaging: A literature review, Med. Image Anal. 73 (2021) 102193.
[4] N. Zeng, H. Li, Z. Wang, W. Liu, S. Liu, F.E. Alsaadi, X. Liu, Deep-reinforcement-learning-based images segmentation for quantitative analysis of gold immunochromatographic strip, Neurocomputing 425 (2021) 173–180.
[5] K. Arulkumaran, M.P. Deisenroth, M. Brundage, A.A. Bharath, Deep reinforcement learning: A brief survey, IEEE Signal Process. Mag. 34 (6) (2017) 26–38.
[6] R. Bellman, Dynamic programming, Science 153 (3731) (1966) 34–37.
[7] Y. Dong, S. Zhang, X. Liu, Y. Zhang, T. Shen, Variance aware reward smoothing for deep reinforcement learning, Neurocomputing 458 (2021) 327–335.
[8] R.S. Sutton, A.G. Barto, Reinforcement Learning: An introduction, MIT Press, 2018.
[9] A.Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: Theory and application to reward shaping, in: ICML, vol. 99, Citeseer, 1999, pp. 278–287.
[10] Y. Dong, X. Tang, Y. Yuan, Principled reward shaping for reinforcement learning via Lyapunov stability theory, Neurocomputing 393 (2020) 83–90.
[11] D. Zhang, C.P. Bailey, Obstacle avoidance and navigation utilizing reinforcement learning with reward shaping, in: Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications II, vol. 11413, SPIE, 2020, pp. 500–506.
[12] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, J. Mach. Learn. Res. 13 (2) (2012).
[13] R.G. Mantovani, A.L. Rossi, J. Vanschoren, B. Bischl, A.C. De Carvalho, Effectiveness of random search in SVM hyper-parameter tuning, in: 2015 International Joint Conference on Neural Networks, IJCNN, IEEE, 2015, pp. 1–8.
[14] Y. Sun, B. Xue, M. Zhang, G.G. Yen, An experimental study on hyper-parameter optimization for stacked auto-encoders, in: 2018 IEEE Congress on Evolutionary Computation, CEC, IEEE, 2018, pp. 1–8.
[15] I. Loshchilov, F. Hutter, Sgdr: Stochastic gradient descent with warm restarts, 2016, arXiv preprint arXiv:1608.03983.
[16] P. Kumar, S. Batra, B. Raman, Deep neural network hyper-parameter tuning through twofold genetic approach, Soft Comput. 25 (2021) 8747–8771.
[17] V. Konda, J. Tsitsiklis, Actor-critic algorithms, Adv. Neural Inf. Process. Syst. 12 (1999).
[18] S. Fujimoto, H. Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: International Conference on Machine Learning, PMLR, 2018, pp. 1587–1596.
[19] L. Pan, Q. Cai, L. Huang, Softmax deep double deterministic policy gradients, Adv. Neural Inf. Process. Syst. 33 (2020) 11767–11777.
[20] E. Wiewiora, G.W. Cottrell, C. Elkan, Principled methods for advising reinforcement learning agents, in: Proceedings of the 20th International Conference on Machine Learning (ICML-03), 2003, pp. 792–799.
[21] L. Pan, Q. Cai, L. Huang, Softmax deep double deterministic policy gradients, Adv. Neural Inf. Process. Syst. 33 (2020) 11767–11777.



**Zhelin Deng** received his bachelor degree from the School of Mathematics and Statistics, Huazhong University of Science and Technology in 2018, and his master degree from the College of Business, City University of Hong Kong, China in 2020. He is currently working towards the Ph.D. degree at School of Artificial Intelligence and Automation, Huazhong University of Science and Technology, Wuhan, China. His research interest covers RL in smart manufacturing.



**Xing Liu** received the B.E. degree from the School of Electrical and Automation Engineering, East China Jiaotong University in 2020, and his master degree from the School of Information Science and Engineering, East China University of Science and Technology, Shanghai, China. He is currently working towards the Ph.D. degree at School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, China. His research interests include RL, imitation learning and robotics.



**Yunlong Dong** received the B.E. degree and Ph.D. degrees from the School of Automation, Huazhong University of Science and Technology, Wuhan, China, in 2017 and 2022. He is currently a postdoctoral researcher in the Department of Automation, Tsinghua University, Beijing, China. His research interests include RL, control theory and robotics.